

DBMS Unit 4 - PL/SQL Programming

UNIT - IV: PL/SQL PROGRAMMING

MSBTE K-Scheme | Diploma CO | SY - Database
Management System

4.1 INTRODUCTION TO PL/SQL

PL/SQL (Procedural Language extension to SQL) is a procedural extension of SQL developed by Oracle. It combines the data manipulation power of SQL with the procedural constructs (loops, conditions, variables) of traditional programming languages, allowing complex business logic to be executed directly within the database.

Advantages of PL/SQL

1. **Integration with SQL** – Allows SQL statements to be used directly within procedural code.
2. **Improved Performance** – Since entire blocks are sent to the server at once, network traffic is reduced (compared to sending SQL statement-by-statement).
3. **Procedural Capability** – Supports loops, conditions, variables — features not available in plain SQL.
4. **Error/Exception Handling** – Provides a structured way to handle runtime errors.
5. **Reusability & Modularity** – Code can be written once as procedures/functions and reused multiple times.
6. **Portability** – PL/SQL programs can run on any platform where Oracle is supported.
7. **Security** – Access to underlying tables can be restricted; users only interact through procedures/functions.

The PL/SQL Block Structure

A PL/SQL program is organized into **blocks**. Each block has up to **four sections**:

```
DECLARE
    -- Declaration Section (optional)
    -- Variables, constants, cursors are declared here
BEGIN
    -- Execution Section (mandatory)
    -- SQL and PL/SQL statements go here
EXCEPTION
    -- Exception Handling Section (optional)
    -- Handles runtime errors
END;
```

Section	Mandatory?	Purpose
DECLARE	No	Declares variables, constants, cursors, and user-defined

BEGIN...END (Execution)	Yes	exceptions Contains the actual SQL/PL-SQL statements to be executed
EXCEPTION	No	Handles errors that occur during execution

Types of Blocks: - **Anonymous Block** - has no name, not stored in the database, executed once. - **Named Block (Subprogram)** - Procedures, Functions, Triggers, Packages — stored permanently in the database and can be reused.

PL/SQL Data Types

Category	Data Types	Description
Scalar	NUMBER, VARCHAR2, CHAR, DATE, BOOLEAN	Single values
Composite	RECORD, TABLE, VARRAY	Collection of multiple values
Reference	REF CURSOR	Points to a storage location (cursor variable)
LOB (Large Object)	BLOB, CLOB, BFILE	Store large unstructured data like images, text

Variable

A **variable** is a named storage location whose value can **change** during program execution. It must be declared before use, in the DECLARE section.

Syntax:

```
variable_name datatype [:= initial_value];
```

Example:

```
DECLARE
    v_name VARCHAR2(20);
    v_marks NUMBER := 0;
BEGIN
    v_name := 'Rahul';
    v_marks := 85;
    DBMS_OUTPUT.PUT_LINE(v_name || ' scored ' || v_marks);
END;
```

Constant

A **constant** is a named storage location whose value **cannot be changed** once assigned, throughout the program's execution. It must be assigned a value at the time of declaration using the CONSTANT keyword.

Syntax:

```
constant_name CONSTANT datatype := value;
```

Example:

```
DECLARE
    pi CONSTANT NUMBER := 3.14159;
```

```
BEGIN
    DBMS_OUTPUT.PUT_LINE('Value of PI is ' || pi);
END;
```

4.2 CONTROL STRUCTURES

Control structures determine the **order** in which statements are executed in a PL/SQL block.

1. Conditional Control

Used to execute a block of statements based on whether a condition is TRUE or FALSE.

a) IF-THEN

```
IF condition THEN
    statements;
END IF;
```

b) IF-THEN-ELSE

```
IF condition THEN
    statements1;
ELSE
    statements2;
END IF;
```

c) IF-THEN-ELSIF

```
IF condition1 THEN
    statements1;
ELSIF condition2 THEN
    statements2;
ELSE
    statements3;
END IF;
```

d) CASE Statement

```
CASE grade
    WHEN 'A' THEN DBMS_OUTPUT.PUT_LINE('Excellent');
    WHEN 'B' THEN DBMS_OUTPUT.PUT_LINE('Good');
    ELSE DBMS_OUTPUT.PUT_LINE('Average');
END CASE;
```

2. Iterative Control (Loops)

Used to execute a set of statements **repeatedly**.

a) Basic (Simple) Loop – executes at least once; requires an explicit EXIT condition.

```
LOOP
    statements;
    EXIT WHEN condition;
END LOOP;
```

b) WHILE Loop – checks the condition **before** each iteration.

```
WHILE condition LOOP
    statements;
END LOOP;
```

c) FOR Loop – executes for a fixed number of iterations based on a range.

```

FOR i IN 1..10 LOOP
    DBMS_OUTPUT.PUT_LINE(i);
END LOOP;

```

3. Sequential Control

Used to alter the normal sequential flow of execution.

- **GOTO statement** – transfers control unconditionally to a labeled statement.

```

GOTO label_name;
...
<<label_name>>
statements;

```

- **NULL statement** – used as a placeholder that does nothing (useful where a statement is syntactically required but no action is needed).

```

IF condition THEN
    NULL; -- do nothing
END IF;

```

4.3 EXCEPTION HANDLING

An **exception** is an error condition that occurs during program execution. PL/SQL provides a structured mechanism to **trap and handle** such runtime errors using the EXCEPTION section, preventing abnormal termination of the program.

Syntax:

```

BEGIN
    -- executable statements
EXCEPTION
    WHEN exception_name1 THEN
        -- handling statements
    WHEN exception_name2 THEN
        -- handling statements
    WHEN OTHERS THEN
        -- handles any other exception
END;

```

1. Predefined Exceptions

These are exceptions that are **automatically raised** by the Oracle server when common errors occur. They already have names and do not need to be declared.

Exception	Raised When
NO_DATA_FOUND	A SELECT INTO statement returns no rows
TOO_MANY_ROWS	A SELECT INTO statement returns more than one row
ZERO_DIVIDE	An attempt is made to divide a number by zero
DUP_VAL_ON_INDEX	An attempt is made to insert a duplicate value in a unique index/column
VALUE_ERROR	An arithmetic, conversion, or size error occurs
INVALID_CURSOR	An illegal cursor operation occurs

LOGIN_DENIED

Invalid username/password used
to log in

Example:

```
BEGIN
  SELECT Marks INTO v_marks FROM Student WHERE Roll_No = 999;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No student found with this Roll No. ');
  WHEN ZERO_DIVIDE THEN
    DBMS_OUTPUT.PUT_LINE('Division by zero not allowed. ');
END;
```

2. User-Defined Exceptions

These are exceptions **explicitly declared and raised by the programmer** for application-specific error conditions that Oracle does not predefine.

Steps: 1. **Declare** the exception in the DECLARE section. 2. **Raise** it explicitly using the RAISE statement inside the BEGIN block. 3. **Handle** it in the EXCEPTION section.

Example:

```
DECLARE
  insufficient_marks EXCEPTION;
  v_marks NUMBER := 30;
BEGIN
  IF v_marks < 35 THEN
    RAISE insufficient_marks;
  END IF;
EXCEPTION
  WHEN insufficient_marks THEN
    DBMS_OUTPUT.PUT_LINE('Student has failed the exam. ');
END;
```

4.4 CURSORS

A **cursor** is a pointer/private work area allocated by Oracle used to process each row returned by a SQL query **individually**, since PL/SQL cannot directly display results of a multi-row query like SQL can.

Implicit Cursor

- **Automatically created** by Oracle whenever a DML statement (INSERT, UPDATE, DELETE) or a single-row SELECT...INTO is executed.
- Named SQL by default.
- Programmer does not need to declare it explicitly.

Cursor attributes: %FOUND, %NOTFOUND, %ROWCOUNT, %ISOPEN

```
BEGIN
  UPDATE Student SET Marks = 90 WHERE Roll_No = 1;
  IF SQL%FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Record updated successfully. ');
  END IF;
END;
```

Explicit Cursor

- **Explicitly declared** by the programmer to handle queries that return **multiple rows**.
- Gives the programmer full control over row-by-row processing.

Steps for using an Explicit Cursor:

1. Declaring

```
CURSOR c_student IS
    SELECT Roll_No, Name FROM Student WHERE Branch = 'CO';
```

2. Opening the Cursor

```
OPEN c_student;
```

3. Fetching a Record from Cursor

```
FETCH c_student INTO v_roll, v_name;
```

4. Closing the Cursor

```
CLOSE c_student;
```

Complete Example:

```
DECLARE
    CURSOR c_student IS SELECT Roll_No, Name FROM Student;
    v_roll Student.Roll_No%TYPE;
    v_name Student.Name%TYPE;
BEGIN
    OPEN c_student;
    LOOP
        FETCH c_student INTO v_roll, v_name;
        EXIT WHEN c_student%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(v_roll || ' - ' || v_name);
    END LOOP;
    CLOSE c_student;
END;
```

Cursor FOR Loops

A **cursor FOR loop** simplifies cursor handling by **automatically** opening, fetching, and closing the cursor — no need to explicitly use OPEN, FETCH, CLOSE.

```
BEGIN
    FOR rec IN (SELECT Roll_No, Name FROM Student) LOOP
        DBMS_OUTPUT.PUT_LINE(rec.Roll_No || ' - ' || rec.Name);
    END LOOP;
END;
```

Parameterized Cursors

A **parameterized cursor** accepts **parameters** at the time it is opened, allowing the same cursor to be reused with different filter values.

```
DECLARE
    CURSOR c_student(p_branch VARCHAR2) IS
        SELECT Roll_No, Name FROM Student WHERE Branch = p_branch;
BEGIN
    FOR rec IN c_student('CO') LOOP
        DBMS_OUTPUT.PUT_LINE(rec.Roll_No || ' - ' || rec.Name);
    END LOOP;
END;
```

4.5 PROCEDURES

A **stored procedure** is a named PL/SQL block that performs a specific task and is **stored permanently** in the database. It can accept parameters and is invoked explicitly using the CALL or EXEC statement (does not return a value directly via its name).

Advantages of Procedures

1. Improves **code reusability** - written once, called multiple times.
2. Improves **performance** - precompiled and stored in the database.
3. Provides **modularity** - complex tasks broken into smaller units.
4. Enhances **security** - users can execute a procedure without direct table access.
5. Reduces **network traffic** - only the procedure call is sent, not multiple SQL statements.

Create Procedure

```
CREATE OR REPLACE PROCEDURE update_marks
(p_roll IN NUMBER, p_marks IN NUMBER)
IS
BEGIN
    UPDATE Student SET Marks = p_marks WHERE Roll_No = p_roll;
    COMMIT;
END;
```

Parameter Modes: | Mode | Description | |—|—| | **IN** | Passes a value into the procedure (default mode, read-only) | | **OUT** | Returns a value from the procedure to the caller | | **IN OUT** | Passes a value in and returns a (possibly modified) value out |

Execute Procedure

```
EXEC update_marks(1, 95);
```

or

```
BEGIN
    update_marks(1, 95);
END;
```

Delete (Drop) a Stored Procedure

```
DROP PROCEDURE update_marks;
```

4.6 FUNCTIONS

A **function** is a named PL/SQL block similar to a procedure, but it **must return a single value** to the caller using the RETURN statement, and can be used directly within SQL expressions.

Advantages of Functions

1. Can be used directly inside SQL statements (e.g., in SELECT).
2. Promotes reusability of calculation logic.
3. Improves performance through precompilation.
4. Makes complex expressions/queries simpler and more readable.

Create Function

```
CREATE OR REPLACE FUNCTION get_grade (p_marks IN NUMBER)
RETURN VARCHAR2
IS
    v_grade VARCHAR2(10);
```

```

BEGIN
    IF p_marks >= 75 THEN
        v_grade := 'Distinction';
    ELSIF p_marks >= 60 THEN
        v_grade := 'First Class';
    ELSE
        v_grade := 'Pass';
    END IF;
    RETURN v_grade;
END;

```

Execute Function

```
SELECT Name, get_grade(Marks) FROM Student;
```

or

```

DECLARE
    v_result VARCHAR2(10);
BEGIN
    v_result := get_grade(85);
    DBMS_OUTPUT.PUT_LINE(v_result);
END;

```

Delete (Drop) a Function

```
DROP FUNCTION get_grade;
```

Procedure vs Function:

Basis	Procedure	Function
Return value	May or may not return a value (via OUT parameter)	Must return exactly one value (via RETURN)
Usage in SQL	Cannot be used directly in SELECT statement	Can be used directly in SELECT statement
Called by	CALL / EXEC statement	Can be called as part of an expression
Purpose	Performs an action/task	Performs computation and returns a result

4.7 DATABASE TRIGGERS

A **trigger** is a special type of stored PL/SQL block that is **automatically executed (fired)** by the database in response to a specific event — such as INSERT, UPDATE, or DELETE — occurring on a table.

Use of Database Triggers

1. **Enforcing complex business rules** that cannot be handled by simple constraints.
2. **Auditing** – automatically logging changes made to a table (who changed what and when).
3. **Maintaining data integrity** – automatically updating related tables when data changes.
4. **Preventing invalid transactions** – e.g., blocking DML operations outside business hours.
5. **Automatic value generation** – e.g., auto-populating timestamp or ID fields.

Types of Triggers

Based on timing: | Type | Description | |—|—| | **BEFORE Trigger** | Fires **before** the triggering DML statement is executed | | **AFTER Trigger** | Fires **after** the triggering DML statement is executed |

Based on level: | Type | Description | |—|—| | **Row-Level Trigger** | Fires **once for each row** affected by the DML statement (uses FOR EACH ROW) | | **Statement-Level Trigger** | Fires **once for the entire statement**, regardless of how many rows are affected |

Based on triggering event: INSERT trigger, UPDATE trigger, DELETE trigger (or a combination).

Create Trigger

```
CREATE OR REPLACE TRIGGER trg_before_insert
BEFORE INSERT ON Student
FOR EACH ROW
BEGIN
    :NEW.Marks := NVL(:NEW.Marks, 0); -- default marks to 0 if NULL
END;
```

Example (Audit Trigger):

```
CREATE OR REPLACE TRIGGER trg_audit_update
AFTER UPDATE ON Student
FOR EACH ROW
BEGIN
    INSERT INTO Student_Audit (Roll_No, Old_Marks, New_Marks,
Changed_On)
VALUES (:OLD.Roll_No, :OLD.Marks, :NEW.Marks, SYSDATE);
END;
```

- :OLD refers to the value **before** the change.
- :NEW refers to the value **after** the change.

Delete (Drop) Trigger

```
DROP TRIGGER trg_before_insert;
```

Quick Revision Points

- **PL/SQL Block:** DECLARE (optional) → BEGIN (mandatory) → EXCEPTION (optional) → END
- **Variable** = value can change; **Constant** = fixed value, declared with CONSTANT
- **Control structures:** IF/CASE (conditional), LOOP/WHILE/FOR (iterative), GOTO/NULL (sequential)
- **Exceptions:** Predefined (NO_DATA_FOUND, TOO_MANY_ROWS, ZERO_DIVIDE...) vs User-defined (declared + RAISE)
- **Cursor:** Implicit (auto, single-row DML/SELECT) vs Explicit (multi-row, DECLARE-OPEN-FETCH-CLOSE)
- **Cursor FOR loop** auto-handles open/fetch/close; **Parameterized cursor** accepts arguments
- **Procedure:** performs action, may not return value, called via EXEC
- **Function:** must RETURN a value, usable inside SQL SELECT statements
- **Trigger:** auto-fires on INSERT/UPDATE/DELETE; BEFORE/AFTER, Row-level/Statement-level; uses :OLD and :NEW